

SPRAWOZDANIE Z PROJEKTU AKADEMICKIEGO

Temat: System Bazy Danych Anime i Postaci (C++)

1. Opis tematu

Projekt polegał na stworzeniu konsolowej aplikacji do zarządzania bazą danych o relacji bohaterów z anime.

System obsługuje trzy rodzaje powiązań:

- Postać < = > Postać (np. rodzina, rywalizacja)
- Anime < = > Postać (np. główny bohater, postać drugoplanowa)
- Anime < = > Anime (np. sequel, historia poboczna)

Dane są trwałe – program umożliwia ich zapis i odczyt z plików tekstowych.

2. Interfejs użytkownika

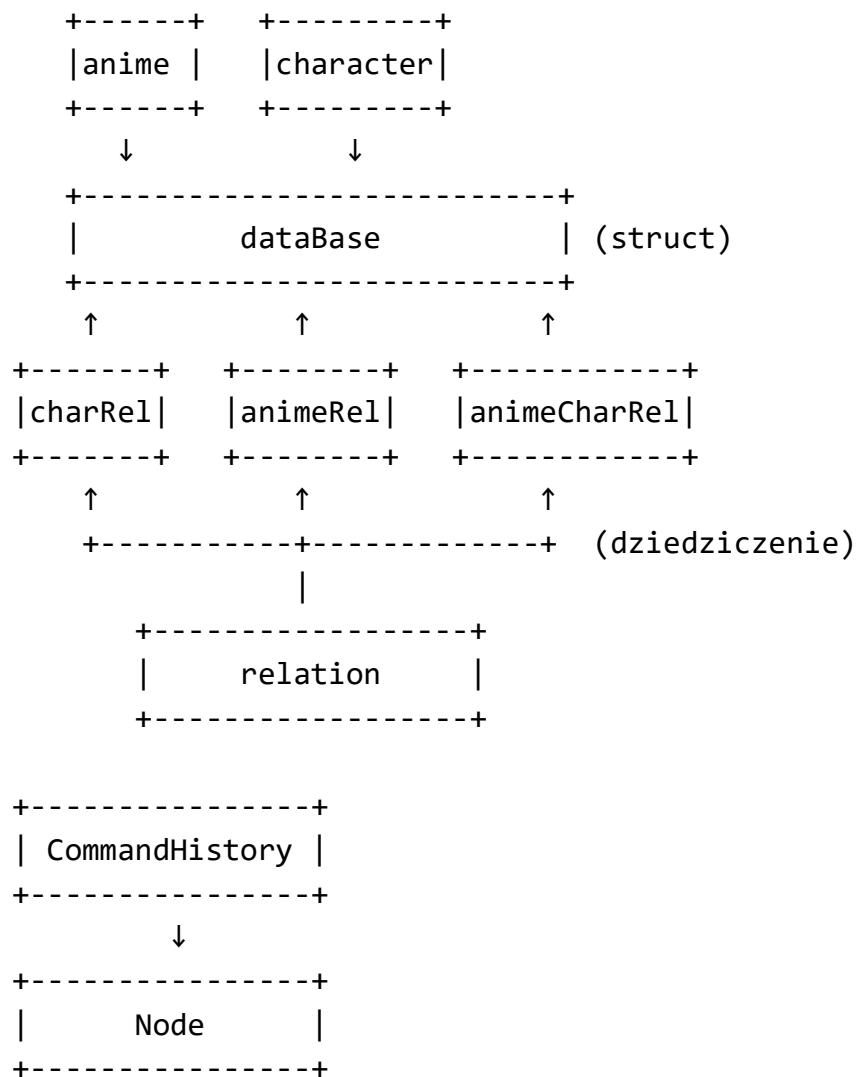
Program działa w trybie tekstowym. Po uruchomieniu wita użytkownika krótkim komunikatem, a pełną listę komend ukrywa pod poleceniem `help`.

Główne komendy systemu:

- **help** – Wyświetla kompletną, sformatowaną listę wszystkich dostępnych komend wraz z ich dokładnymi parametrami.
- **create character [fName] [mName] [lName]** – Rejestruje nową postać w systemie (drugie imię i nazwisko są opcjonalne. Priorytetyzuje nazwisko od drugiego imienia jak są podane 2 wartości.).
- **create anime [name]** – Tworzy nowy tytuł serii anime w bazie danych.
- **update character [ID|Name] [fName|mName|lName|bio] [new_value]** – Aktualizuje wybrane pole tekstowe w strukturze konkretnej postaci.
- **update anime [ID|Name] name [new_value]** – Modyfikuje oficjalną nazwę wybranej serii anime.
- **remove character [ID|Name]** – Usuwa postać z bazy danych oraz kaskadowo czyści wszystkie relacje, w których ten bohater brał udział.
- **remove anime [ID|Name]** – Usuwa wybrane anime oraz kaskadowo kasuje powiązane z nim relacje z innymi seriami lub postaciami.
- **connect anime to character [anime] [char] [desc]** – Tworzy powiązanie między serią anime a postacią (np. przypisanie roli).
- **connect character to character [char1] [char2] [desc]** – Definiuje relację interpersonalną między dwoma bohaterami.
- **connect anime to anime [anime1] [anime2] [desc]** – Wiąże ze sobą dwa tytuły anime (np. w celu oznaczenia sequela).
- **show characters [optional: id|name]** – Wyświetla całą bazę postaci lub filtruje widok względem imienia.
- **show anime [optional: id|name]** – Wyświetla kompletną listę zarejestrowanych serii anime lub szczegóły jednej wybranej.
- **show relations [optional: char|anime]** – Przegląd ogólny relacji lub filtrowanie powiązań wewnątrzgatunkowych (postać-postać, anime-anime).
- **show relations anime_char by [anime|character] [Name|ID]** – Dedykowane okno wyszukiwania relacji. Pozwala na wyciągnięcie danych poprzez wskazanie, czy szukamy powiązań dla konkretnego tytułu serii, czy dla konkretnego bohatera.
- **history** – Wywołuje z pamięci dynamicznej (struktury stosu) listę wszystkich komend wprowadzonych przez użytkownika od momentu uruchomienia aplikacji.
- **save** – Serializuje aktualny stan bazy z pamięci RAM do odpowiednich plików tekstowych na dysku.

- **load** – Czyści pamięć podręczną, wczytuje dane z plików i automatycznie synchronizuje liczniki unikalnych ID (wykrywając najwyższe wartości wejściowe).
- **exit** – Bezpiecznie zwalnia zasoby i kończy działanie programu.

3. Diagrama klas



4. Struktury danych

W projekcie połączono dwa podejścia do zarządzania pamięcią:

- **Kontenery standardowe (`std::vector`):** Użyte w głównej bazie danych do przechowywania dynamicznych list postaci, anime i relacji. Zapewniają bardzo szybki dostęp do danych.
- **Własna dynamiczna lista jednokierunkowa:** Zaimplementowana od zera na bazie wskaźników (`Node* next`) do obsługi historii komend. Działa jak stos, nowe komendy są dodawane na początek przy użyciu operatora `new`. Destruktor klasy dba o ręczne zwolnienie całej pamięci pętlą z operatorem `delete`.

5. Wnioski i napotkane problemy

Napotkane problemy i rozwiązania:

1. **Problem z dublowaniem ID po wczytaniu bazy:** Po użyciu komendy load, licznik nowych ID resetował się, co powodowało błędy przy tworzeniu nowych obiektów.
 - a. *Rozwiązanie:* Dodano tryb mode : :SET do obsługi metadanych. Podczas ładowania plików program sprawdza najwyższe istniejące ID i automatycznie aktualizuje plik meta.txt.
2. **Gubienie wyrazów po spacji:** Standardowy podział tekstu psuł imiona wielocłonowe.
 - a. *Rozwiązanie:* Napisano funkcję parseArguments, która ignoruje spacje, jeśli tekst znajduje się wewnątrz cudzysłowu.

Uzyskana wiedza:

- Praktyczna nauka **zarządzania pamięcią dynamiczną** w C++ (tworzenie struktur wskaźnikowych, unikanie wycieków pamięci poprzez poprawne destruktory).

6. Stan repozytorium Git

Dane bazy danych:

- Anime.txt
- Anime_char_relations.txt
- Anime_relations.txt
- Char_relations.txt
- Characters.txt
- Meta.txt

Klasy:

- Class.cpp
- Class.h

Funkcje operacji nad bazą danych:

- Func_db.cpp
- Func_db.h

Funkcje operacji z plikami:

- Func_file.cpp
- Func_file.h

Funkcje do operacji z komunikatami użytkownika:

- Func_ui.cpp
- Func_ui.h

Global.cpp - dodatkowe enum

Main.cpp - main